Challenge 2 — Home Handyman Portfolio

COMP5450 Mobile Programming · Group 5

Course requirements addressed

• **Flutter** — full project runnable from Android Studio / IntelliJ IDEA (Flutter plugin).
• **Firebase** — portfolio lists and contact submissions use **Cloud Firestore** (`services`, `projects`, `reviews`, `contact_messages`). First launch seeds empty collections from mock data.
• **README.pdf** — configuration, exact structure, screenshots, and public GitHub link (this document).
• **D2L ZIP** — submit Dart sources, screenshot images, and README.pdf (see course instructions).
• **Zoom presentation** — per D2L schedule.

Overview

A Flutter portfolio app for a fictional home handyman service. Six screens — Home, Services, Portfolio, About, Reviews, and Contact — use a Material 3 bottom navigation bar. **Firebase Core** initializes at startup; **FirestorePortfolioRepository** loads portfolio data. If Firebase is not configured (missing `google-services.json`), the app falls back to `MockPortfolioRepository` so the UI still runs.

App description

• **Home** — Hero banner with Call / Book / Portfolio actions, stat chips, featured services from Firestore (or mock).
• **Services** — Grid of service categories with icons and descriptions from Firestore (or mock).
• **Portfolio** — Project cards with categories, descriptions, and network header images (URLs in mock seed data).
• **Reviews** — Average rating and testimonial cards from Firestore (or mock).
• **Contact** — Business info and validated form; submissions written to Firestore `contact_messages` (or mock in tests / fallback).
• **About** — From the app bar info icon: bio, certifications, service area.

Architecture

`PortfolioRepository` defines `getServices()`, `getProjects()`, `getReviews()`, and `submitContact()`. `FirestorePortfolioRepository` implements these with Firestore; `MockPortfolioRepository` is used for `AppEntry(forceMock: true)` in tests and as a runtime fallback. `PortfolioScope` (InheritedWidget) exposes lists + repository to screens.

Exact project structure (Dart sources)

```
lib/
main.dart
app_entry.dart
handyman_app.dart
```

```
app_shell.dart
portfolio_scope.dart
data/
portfolio_repository.dart
mock_portfolio_repository.dart
mock_data.dart
firestore_serializers.dart
firestore_seeder.dart
firestore_portfolio_repository.dart
models/
service.dart
project.dart
review.dart
screens/
home_screen.dart
services_screen.dart
portfolio_screen.dart
about_screen.dart
reviews_screen.dart
contact_screen.dart
widgets/
hero_banner.dart
stat_chip.dart
service_card.dart
project_card.dart
review_card.dart
test/
widget_test.dart
pubspec.yaml
android/ … ios/ … web/ … (Flutter platform scaffolding)
(Also on GitHub: docs/FIREBASE_SETUP.md, firestore.rules, presentation/)
```

How to configure and run

IDE: Android Studio or IntelliJ IDEA — install Flutter & Dart plugins, then **File** → **Open** this project folder (contains `pubspec.yaml`).

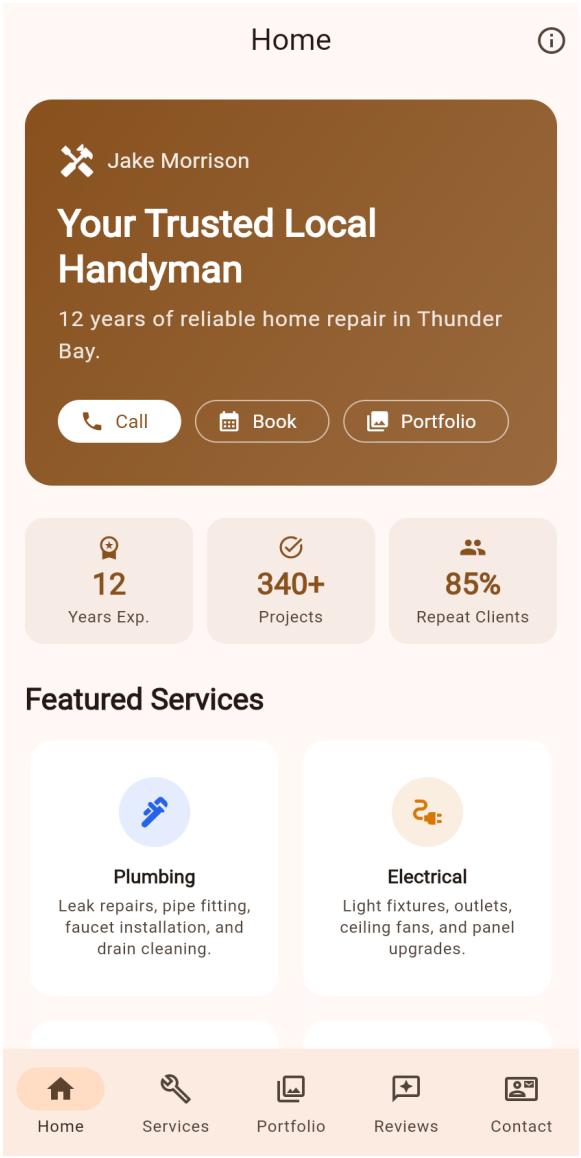
1. Install Flutter 3.32+ and add to PATH.
2. Clone: `git clone https://github.com/achenachena/handyman-portfolio.git`
3. `cd handyman-portfolio`
4. `flutter pub get`
5. Add `android/app/google-services.json` from Firebase Console; enable Firestore (see Firebase setup in the GitHub repo or course materials).
6. `flutter run` — choose an Android emulator or Chrome.

Testing

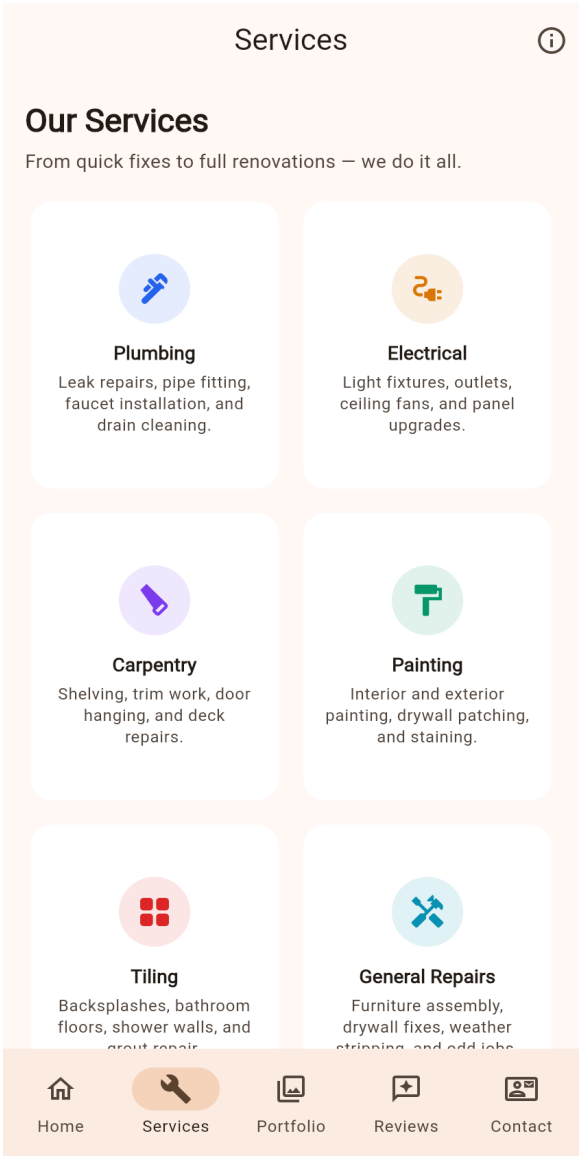
`flutter analyze` — zero issues. `flutter test` — 7 widget tests (mock repository). Screenshots below were captured from the running app (web or emulator).

Screenshots

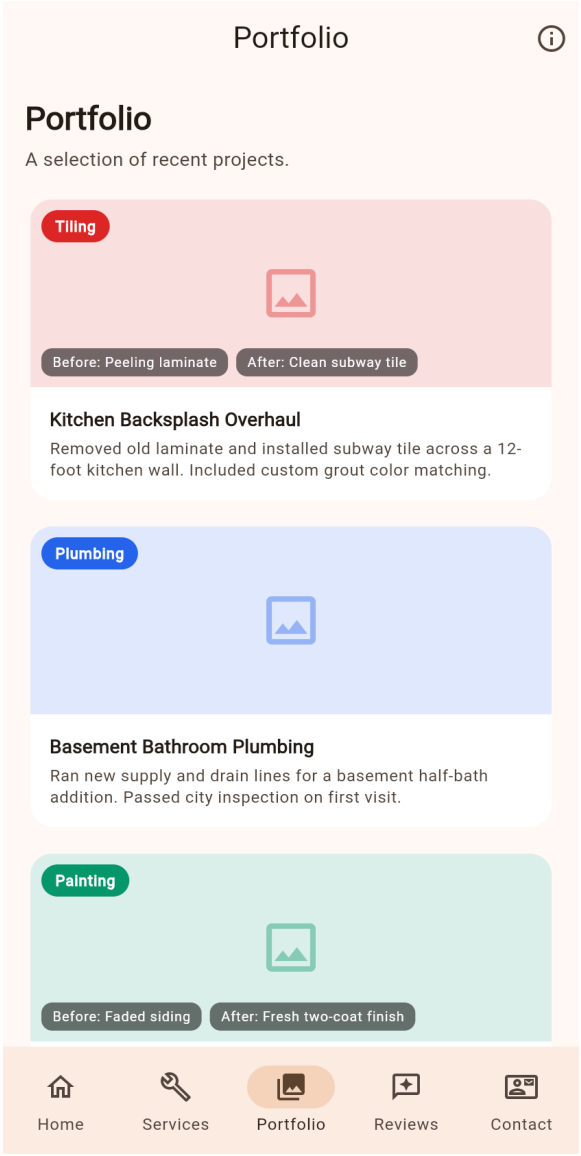
Home screen



Services screen



Portfolio screen



D2L ZIP (requirement #5): submit only all `.dart` files under `lib/` and `test/`, screenshot images, and `README.pdf`. Build the archive with `package_d2l_zip.sh`. The full Flutter project stays on GitHub (requirement #4).

Public GitHub repository

https://github.com/achenachena/handyman-portfolio

Team — Group 5

(see D2L for member list)